

Varsha - 5

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1 - CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL 1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25 100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	VARSHA YARDHINI.S	Reg.No.:	192511488
Department:	BE, CSE	Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

CPU utilization must be below 15%
Multiple sensors generate interrupts every 1 ms
Use vectored interrupt handling
Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

High-speed storage requires throughput > 3 GB/s
Must use NVMe over PCIe
CPU should not handle bulk data transfer
Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

6 peripherals using USB, 2 high-speed devices using Thunderbolt
Avoid interrupt starvation
Use hardware and software interrupts
Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

Continuous data stream at 500 MB/s
Minimal CPU intervention required
Must compare DMA transfer vs interrupt-driven I/O
Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

Support nested interrupts
Maximum interrupt latency < 10 μ s
Use vectored interrupts for high-priority devices
Use software interrupts for background tasks

81%

Code of Conduct:

I Varsha Vaedhini.S / 192511488 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: §. Vaedhini

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	2	1.5	1	1
Q2	2	2	1.5	1	1
Q3	2.5	2.5	2	1.5	1
Q4	2.5	2	2	1.5	0.5
Q5	2	1.5	2.5	1	0.5

RUBRICS FOR CALCULATION:

40.5

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

ANSWER :

The system must handle **multiple sensor interrupts every 1 ms** while keeping **CPU utilization below 15%**. Therefore, the design should minimize interrupt-processing overhead and avoid unnecessary CPU involvement. The required solution must use **vectored interrupt handling** and select an I/O addressing method for minimum latency.

Constraint	Design Implication	Priority
CPU utilization < 15%	Interrupt Service Routines (ISRs) must be short and efficient	Critical
Multiple interrupts every 1 ms	Fast identification and servicing of the interrupt source is required	Critical
Vectored interrupt handling	Each sensor should have a predefined interrupt vector/ISR	Mandatory
Optimal I/O latency	Device registers should be accessed with minimum addressing overhead	High

Selected Mechanism: Vectored Interrupts + Memory-Mapped I/O

Vectored interrupt handling is selected because the interrupt controller directly provides the address of the appropriate ISR. The CPU does not need to repeatedly identify which sensor generated the interrupt. This reduces interrupt-handling overhead and improves response time.

For the I/O addressing method, **memory-mapped I/O (MMIO)** is selected. Sensor registers are mapped into the processor's normal address space, allowing the CPU to access them using standard load/store instructions. This provides a direct and efficient path for accessing device status and data registers.

Operation of the Proposed System

1. A sensor generates an interrupt.
2. The interrupt controller receives and prioritizes the request.
3. A corresponding interrupt vector directs the CPU immediately to the correct ISR.
4. The ISR reads the required sensor status/data through MMIO.
5. The ISR performs only essential real-time operations.
6. Additional processing is deferred until after the critical interrupt service is completed.
7. The CPU returns quickly to normal execution.

How the Design Meets the Constraints

Requirement	Design Decision	Result
CPU utilization below 15%	Short, dedicated ISRs with minimal processing	Reduces CPU time spent per interrupt
Multiple interrupts every 1 ms	Interrupt controller with prioritization	Handles frequent sensor requests systematically
Vectored interrupt handling	Separate vector for each sensor/interrupt source	Eliminates interrupt-source identification delay
Low I/O latency	Memory-mapped I/O	Direct register access using normal memory instructions

Justification

The most important design decision is to keep the **ISR extremely short**. With interrupts arriving every 1 ms, lengthy processing inside each ISR could quickly consume CPU time and violate the 15% utilization constr

aint. Vectored interrupts reduce dispatch overhead, while MMIO provides direct access to sensor registers.

Thus, the proposed mechanism combines:

Sensors → Priority Interrupt Controller → Vectored ISR → MMIO Register Access → Quick Return

Conclusion

The proposed I/O communication mechanism satisfies all the given constraints by combining **vectored interrupt handling with memory-mapped I/O**. Vectored interrupts provide fast identification of each sensor, while short ISRs reduce CPU overhead. MMIO enables efficient device-register access, making the overall design suitable for a real-time embedded system where **low latency and controlled CPU utilization are critical**.

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

ANSWER :

Problem Understanding

The I/O subsystem must support a **high-speed NVMe storage device** with throughput greater than **3 GB/s**. The CPU should only manage commands and completion handling, while the **bulk data transfer must occur without continuous CPU involvement**. Therefore, the most suitable design combines **NVMe over PCIe, DMA-based data transfer, and interrupt-based completion notification**.

Constraint Analysis

Constraint	Design Requirement
Throughput > 3 GB/s	Use a high-bandwidth PCIe link suitable for NVMe transfers
NVMe over PCIe	Connect the storage device through the PCIe/NVMe protocol stack
CPU not involved in bulk transfer	Use DMA for direct data movement between NVMe and system memory
Completion notification required	Generate an interrupt when the DMA/NVMe operation is completed

Proposed I/O Subsystem

CPU → NVMe Command Queue → PCIe → NVMe SSD

**** ↓ ****

System Memory ←—— DMA Transfer ——→ NVMe SSD

**** ↓ ****

**** Interrupt Controller → CPU ****

The CPU initializes the operation by placing an I/O command in the NVMe command queue and providing the memory buffer information. After that, the **DMA engine transfers the bulk data directly between the NVMe device and RAM**.

Working of the System

1. The CPU prepares the NVMe command and identifies the memory buffer.
2. The command is sent to the NVMe SSD through the **PCIe interface**.
3. The NVMe controller processes the request.
4. The **DMA controller transfers bulk data directly between the SSD and system memory**.
5. During this transfer, the CPU is free to execute other tasks.
6. Once the transfer is completed, an **interrupt notification** is generated.
7. The CPU handles the completion interrupt and updates the I/O status.

Performance Analysis

Performance Aspect	Proposed Design Impact
Throughput	PCIe + NVMe provides a high-bandwidth path capable of exceeding 3 GB/s
CPU utilization	Reduced because bulk transfers are handled by DMA
Data transfer efficiency	Direct NVMe-to-memory transfer reduces unnecessary CPU copying
Latency	Low PCIe/NVMe communication latency with asynchronous DMA operation
CPU availability	CPU remains available for application and system tasks during transfer
Completion handling	Interrupt notification eliminates continuous polling
Scalability	Queue-based NVMe architecture can support multiple outstanding I/O operations

Final Design Justification

The proposed subsystem uses **NVMe over PCIe for high-speed communication, DMA for bulk data movement, and interrupts for completion notification**. This directly satisfies all the constraints: PCIe provides the required throughput, DMA removes the CPU from the bulk data path, and interrupts inform the CPU only after the operation is completed.

Conclusion

The optimal design is:

CPU (Command Setup) → NVMe over PCIe → DMA ↔ System Memory → Interrupt on Completion

This architecture achieves **high throughput above 3 GB/s**, minimizes CPU involvement, and provides efficient completion handling, making it suitable for high-performance storage systems.

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

ANSWER :

Problem Understanding

The system must support **six USB peripherals and two high-speed Thunderbolt devices** while ensuring that no device or interrupt source is permanently delayed. The architecture therefore needs **separate high-speed communication paths, fair bus arbitration, and controlled interrupt handling using both hardware and software interrupts.**

Constraint Analysis

Constraint	Design Requirement
6 USB peripherals	USB host controller with sufficient ports/hub support
2 Thunderbolt devices	Dedicated high-speed Thunderbolt controller/interface
Avoid interrupt starvation	Fair interrupt scheduling with priority control and aging
Hardware + software interrupts	Hardware interrupts for device events; software interrupts for deferred processing
Fair bus arbitration	Round-robin or weighted round-robin arbitration between competing devices

Proposed Design Strategy

a) USB Communication

The six USB peripherals are connected through a **USB host controller**, using hubs where required. The host controller manages device communication and generates hardware interrupts for events such as transfer completion or device status changes.

b) Thunderbolt Communication

The two high-speed devices use a dedicated **Thunderbolt controller**, allowing high-bandwidth traffic to enter the system without competing directly with low-speed USB transactions at the device-management level.

c) Fair Bus Arbitration

A **round-robin arbitration mechanism** is used when multiple controllers request access to the system interconnect. After one controller receives service, priority moves to the next requester.

This prevents a continuously active high-speed device from permanently blocking USB traffic.

How the Architecture Meets the Constraints

Requirement	Proposed Mechanism	Result
6 USB devices	USB host controller + hub support	All USB peripherals can communicate

2 Thunderbolt devices	Dedicated Thunderbolt controller	Supports high-speed communication
Avoid interrupt starvation	Priority aging + short ISRs	Every pending device eventually receives CPU service
Hardware interrupts	Immediate device-event notification	Fast response to critical events
Software interrupts	Deferred processing	Reduces ISR execution time
Fair bus access	Round-robin arbiter	Prevents one controller from monopolising the bus

Conclusion

The proposed architecture combines **USB and Thunderbolt controllers with a fair round-robin bus arbiter and a two-stage interrupt mechanism**. Hardware interrupts provide immediate response, while software interrupts handle deferred processing. **Priority aging prevents interrupt starvation**, and round-robin arbitration ensures that both USB and high-speed Thunderbolt traffic receive fair access to the shared system bus.

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

ANSWER :

Problem Understanding

The system must continuously acquire data at **500 MB/s** while keeping CPU involvement minimal. Since the device produces a sustained data stream, handling every transfer through individual CPU-controlled interrupts would create significant overhead. The design therefore uses **memory-mapped I/O (MMIO) for device control/status registers** and evaluates **DMA versus interrupt-driven I/O** for the actual data transfer.

Constraint Analysis

Constraint	Design Requirement
Continuous stream at 500 MB/s	Sustained high-bandwidth transfer to memory
Minimal CPU intervention	Bulk data movement must occur without CPU copying
Compare DMA vs interrupt-driven I/O	Select the method based on throughput and CPU overhead
Memory-mapped I/O	Device registers accessed using normal memory load/store operations

DMA vs Interrupt-Driven I/O

Factor	DMA Transfer	Interrupt-Driven I/O
Data movement	DMA controller transfers blocks directly to RAM	CPU services each I/O event and moves/manages data
CPU involvement	Minimal after transfer setup	High for continuous high-rate transfers
500 MB/s suitability	Highly suitable	Can create excessive interrupt overhead
Throughput	High and sustained using block/buffer transfers	Limited by ISR frequency and CPU processing time
Latency	Small setup overhead; efficient for large continuous blocks	Fast event response, but repeated interrupts add overhead
Best use	Continuous bulk acquisition	Low-rate or event-driven data

Proposed Architecture

Acquisition Device



MMIO Registers — Control / Status / DMA Configuration



DMA Controller

↓ *(direct bulk transfer)*

System RAM – Circular / Ping-Pong Buffers



Application / Processing Unit

Interrupt → CPU only for buffer completion, errors, or exceptional events

Working of the Proposed System

1. The CPU configures the acquisition device through **memory-mapped control registers**.
2. DMA descriptors are created for multiple RAM buffers.
3. The acquisition device continuously sends data at **500 MB/s**.
4. The DMA controller transfers data directly into RAM without CPU copying each data block.
5. **Circular or ping-pong buffers** allow one buffer to be filled while another is processed.
6. An interrupt is generated only when a buffer is completed or when an error occurs.
7. The CPU processes completed buffers and prepares the next transfer, maintaining continuous acquisition.

Performance Impact

Metric	Interrupt-Driven I/O	Proposed DMA-Based Design
CPU utilization	High due to frequent ISRs	Low; CPU mainly performs setup and buffer management

Sustained throughput	May degrade as interrupt rate increases	Better suited to maintain 500 MB/s
Data movement	CPU-dependent	Direct device-to-memory transfer
Interrupt overhead	Frequent	Reduced to buffer-level notifications
Risk of data loss	Higher if CPU cannot service events quickly	Lower with properly sized DMA buffers
Overall suitability	Not preferred for continuous bulk acquisition	Preferred solution

Conclusion

The recommended system is:

**Device Control → MMIO Registers → DMA Configuration
Continuous Data Stream → DMA → Circular/Ping-Pong RAM Buffers → Processing
Buffer Complete/Error → Interrupt → CPU**

This design is better suited for a continuous **500 MB/s** stream because DMA provides sustained data transfer with minimal CPU intervention, whereas a purely interrupt-driven approach would increase CPU overhead and reduce the ability to maintain continuous throughput.

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu\text{s}$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

ANSWER :

Problem Understanding

The system must handle interrupts with a **maximum latency below $10 \mu\text{s}$** , support **nested interrupts**, provide rapid response for **high-priority devices using vectored interrupts**, and move non-critical background work to **software interrupts**. The design should therefore separate time-critical interrupt handling from deferred processing.

Constraint Analysis

Constraint	Design Requirement
Nested interrupts	A higher-priority interrupt must be allowed to pre-empt a lower-priority ISR
Latency $< 10 \mu\text{s}$	Fast interrupt detection, priority resolution and ISR dispatch are required
Vectored high-priority interrupts	Interrupt controller must directly provide the ISR address/vector
Software interrupts for background tasks	Non-critical processing must be deferred outside the time-critical hardware ISR

Working of the Proposed Design

1. A device generates a hardware interrupt.
2. The interrupt controller determines its priority.
3. For high-priority devices, the controller provides the **interrupt vector**, allowing the CPU to jump directly to the correct ISR.
4. The ISR performs only essential operations: acknowledge the interrupt, capture critical data/status, and clear the interrupt source.
5. If a higher-priority interrupt occurs while a lower-priority ISR is executing, **nested interrupt handling** allows the higher-priority ISR to pre-empt it.
6. Longer or non-critical processing is postponed by triggering a **software interrupt**.
7. After the hardware ISR completes, the CPU services the deferred software interrupt and performs background processing.

Meeting the $< 10 \mu\text{s}$ Latency Requirement

Latency Component	Design Strategy
Interrupt source identification	Vectored interrupts eliminate lengthy polling
Priority selection	Hardware interrupt controller performs priority resolution
ISR dispatch	Direct jump to the ISR using the interrupt vector
ISR execution	ISR is kept short and limited to critical operations
Nested interrupts	Higher-priority events can pre-empt lower-priority processing
Background processing	Deferred to software interrupts

Design Justification

The main design principle is:

Critical work → Hardware Vectored ISR

Non-critical work → Software Interrupt

Vectored interrupts reduce dispatch delay, while priority-based nesting ensures that urgent devices are not blocked by lower-priority ISRs. Software interrupts prevent lengthy background operations from increasing hardware interrupt latency.

Conclusion

The proposed mechanism combines **priority-based nested interrupts, vectored ISR dispatch for high-priority devices, and software interrupts for deferred background tasks**. This structure minimises critical-path latency and supports rapid servicing of urgent events, while keeping non-critical processing separate. With appropriate hardware timing and short ISR

implementation, the system can be engineered and validated to meet the **maximum interrupt latency requirement of less than 10 μ s**.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------